

REC Guardian

AI-Powered Renewable Energy Certificate Fraud Detection & Forensic Intelligence Platform

Theme: Renewable Energy Intelligence

Team: KHATRON KE KHILADI

Name

Role

Varun Backend & AI/ML Engineer

Kevin Blockchain & Security Engineer

Dhruv Frontend & Product Engineer

1. Problem Statement

A Renewable Energy Certificate (REC) represents a claim: *"this much renewable electricity was generated."* That claim passes through many hands — generator, meter, issuer, trader, buyer — before it is finally redeemed.

At every handoff, something can go wrong:

- A generator **claims more energy than the meter actually recorded.**
- The **same generation record or document is submitted twice.**
- **Supporting evidence (documents/reports) is reused** across unrelated claims.
- Certificates are **transferred in suspicious circular patterns** between accounts.
- As certificate volume grows, **manual review simply cannot scale** to catch these patterns.

Existing REC systems are good at checking whether a submission is *complete and properly formatted* — but they are not built to continuously ask whether a submission *behaves* like genuine, consistent activity. That gap is where fraud goes unnoticed.

2. Proposed Solution

REC Guardian is a smart "second layer of checking" that sits alongside the official REC system.

Think of it like a fraud-detection system a bank uses for suspicious transactions — it doesn't replace the bank, it just watches every transaction and flags anything unusual for a human to review.

In the same way, REC Guardian:

1. **Watches every certificate request** as it comes in.
2. **Compares it against the plant's own history** — does this look normal for them?
3. **Checks the numbers against physical reality** — does the claimed generation even make sense for this plant's capacity?
4. **Checks if the same document or data was used before.**
5. **Looks at who is trading with whom** — to catch suspicious circular or clustered activity.
6. Combines everything into **one clear, explainable risk score** — not just "fraud/not fraud," but *why* it's risky.
7. **Locks the record's history in a tamper-proof ledger**, so if it's flagged, investigators can trust the evidence they're looking at.

It never accuses anyone of fraud automatically — it flags things as **"needs review"** or **"high risk"**, and a human regulator or auditor makes the final call.

3. Approach

Our approach is built on one core idea: **Detect → Explain → Investigate → Preserve Evidence.**

Step

What Happens

Detect	Rule engine + Machine Learning (Isolation Forest) + Graph analysis each independently scan every request for a different <i>type</i> of red flag — deterministic violations, statistical anomalies, and relationship patterns.
Explain	Every risk score comes with a breakdown of exactly which checks contributed to it, and by how much — never a single unexplained number.
Investigate	Flagged requests open a structured case file for a human investigator, with all evidence in one place.
Preserve Evidence	Every certificate event and document hash is recorded in a tamper-evident ledger, so nothing can be silently altered after the fact.

We deliberately avoid using AI or blockchain as buzzwords — each technology solves one specific, named problem in this pipeline, and nothing more.

4. Key Features

- **✓ Meter vs. Claim Mismatch Detection** — flags generation claims that don't match metered data or plant capacity.
- **✓ Duplicate & Document Reuse Detection** — hash-based checks catch resubmitted records and reused evidence files instantly.
- **✓ AI-Powered Anomaly Detection** — Isolation Forest learns each plant's normal behavior and flags statistical deviations.
- **✓ Graph-Based Relationship Analysis** — detects suspicious clusters, repeated counterparties, and circular certificate transfers.
- **✓ Explainable Risk Scoring** — every score is broken down into specific, human-readable reasons (not a black box).
- **✓ Tamper-Evident Ledger** — SHA-256 hash-chain preserves the full certificate lifecycle so records can't be quietly changed.
- **✓ Document Integrity Verification** — recomputes and compares hashes at audit time to detect modified evidence.
- **✓ Investigation Dashboard** — one unified view of risk, evidence, entity connections, and lifecycle history for investigators.

-  **Role-Based Dashboards** — tailored views for Generators, Regulators, Auditors, and Administrators.
 -  **Human-in-the-Loop Decisions** — the system never auto-labels anything as "fraud"; only flags risk for human review.
-

5. Technology & Implementation Plan

Technology Stack

Layer	Technology
Frontend	React / Next.js, TypeScript, Tailwind CSS
Backend	Python, FastAPI
Database	PostgreSQL, Supabase
Machine Learning	Scikit-learn (Isolation Forest)
Graph Analysis	NetworkX (Neo4j as stretch goal)
Ledger	SHA-256 hash-chain (tamper-evident, append-only)
Storage	S3-compatible object storage, GCP - bucket
Authentication	JWT-based auth

Deployment Docker

System Flow

Data Sources → Secure Ingestion → Validation → Duplicate Detection → Rule Engine
→ Feature Engineering → ML Anomaly Detection → Graph Analysis → Risk Fusion Engine
→ Explainable Risk Score → Decision Engine (Approve / Review / Hold)
→ Certificate Lifecycle → Tamper-Evident Ledger → Investigation Dashboard

Implementation Plan

Phase	Deliverables
Day 1	Database schema, backend APIs, authentication, generation ingestion, rule engine, ML prototype
Day 2	Risk fusion engine, ledger implementation, dashboards, investigation page, graph module, integration
Final Hours	Demo dataset, UI polish, end-to-end testing, presentation rehearsal

Build Priority

- **Must Have:** Authentication, generation submission, duplicate/meter/capacity checks, anomaly detection, risk scoring, regulator dashboard, certificate lifecycle, ledger verification, investigation page.
- **Should Have:** Graph visualization, document hashing, detailed risk breakdown UI.
- **Nice to Have:** Neo4j integration, advanced ML models, full Hyperledger deployment.

The system remains fully demoable even with only the "Must Have" components complete.
